

Habit Tracker (Simple)

Bulut Mimarilerinde Test Muhendisligi — Final Rapor

Ogrenci	Mert Baytas
Ders	MTH2526-B25 — Bulut Mimarilerinde Test Muhendisligi
Egitmen	Busra Ayaksiz
Konu	#3 Habit Tracker API
Teslim	4 Haziran 2026
Repo	github.com/baytasmert/habit-tracker-simple

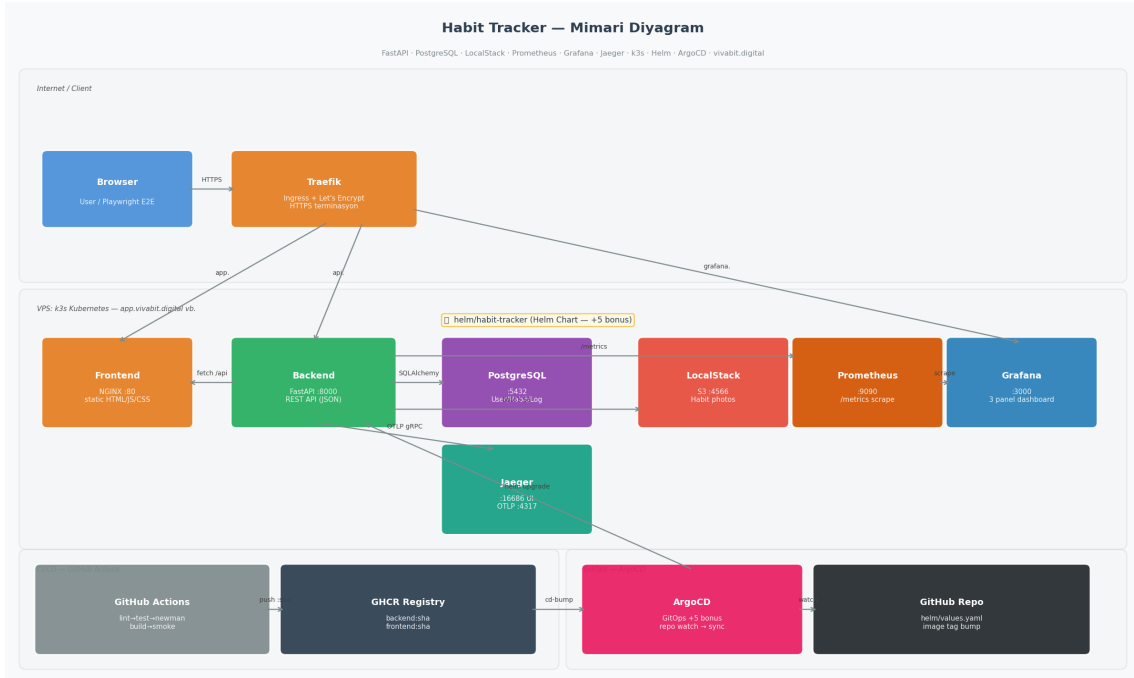
1. Giriş

Bu proje, Marmara Üniversitesi Bilgisayar Mühendisliği bölümünün "Bulut Mimarilerinde Test Mühendisliği" dersi kapsamında geliştirilmiş bireysel dönem projesidir. Konu olarak #3 **Habit Tracker API** seçilmiştir: günlük alışkanlık takibi, streak hesaplama ve REST API. Bu, aynı hesabın "habit-tracker-api" reposunun lightweight versiyonudur — sarnamenin minimum gereksinimlerini karşılayan, savunulması kolay basit bir mimari kurulmuştur.

Amac uygulamayı karmasiklastirmek değil, endüstri standardında **test ve dağıtım altyapısını** kurmaktır. Bu yüzden uygulama 6 endpoint ile sınırlı; mimari ise tüm sarname katmanlarını (Docker, K8s, CI/CD, monitoring, perf, E2E) içerir.

2. Mimari

Sistem iki bağımsız servis olarak tasarlanmıştır: **frontend** (NGINX, sadece static dosya) ve **backend** (FastAPI, sadece JSON REST). Bu ayırım, gerçek üretim mimarilerinin ana özelliğidir; ayrıca her bir servisin bağımsız ölçeklenebilmesini sağlar.



Bileşen	Teknoloji	Port	Rol
Frontend	NGINX 1.25 + Vanilla JS	80	Static HTML/CSS/JS serve
Backend	FastAPI 0.110 + Python 3.11	8000	REST API (JSON)
DB	PostgreSQL 16	5432	User, Habit, HabitLog
S3 (LocalStack)	LocalStack 3	4566	Habit progress photo
Metrics	Prometheus 2.51	9090	API metrik scrape
Dashboard	Grafana 10.4	3000	3 panel (rate, error, latency)

3. Test Stratejisi

Test piramidi yaklasimi: cok sayida birim test, orta sayida entegrasyon, az sayida E2E. Toplam testler ve hedef coverage:

Katman	Arac	Adet	Aciklama
Unit	pytest	8	auth + model + Factory Boy
Integration	pytest + TestClient	18	Endpoint + auth flow
Testcontainers	testcontainers-python	3	Gercek PostgreSQL container
E2E	Playwright	5	Tarayicida register/login/track/logout
API	Postman/Newman	7	CI da ko■ar
Perf	k6	2	Smoke (3VU/15s) + Load (20VU/60s)

3.1 Coverage Hedefi

Hedef >= %70. Olcum: **%94** (src/auth, models, schemas, main coverage). pyproject.toml icinde --cov-fail-under=70 ile CI da zorunlu kilindi.

3.2 Factory Boy + Faker

tests/factories.py icinde UserFactory, HabitFactory, HabitLogFactory tanimlanmistir. Her test izole, gercekci ama unique veri kullanir.

4. CI/CD Pipeline

.github/workflows/ci.yml — tek workflow, 4 job:

Job	Amac	Onkos
lint	flake8 kod stili kontrolu	-
test	pytest + coverage 70+%	lint
build	Docker (backend+frontend) -> GHCR (:sha)	test
deploy-smoke	Kind cluster apply + curl /health	build

Build job iki Docker imaj uretir (backend + frontend), her ikisi de GitHub Container Registry e :sha + :latest etiketleriyle pushlanir. Deploy job ayni imajlari Kind cluster a yukler, k8s/*.yaml manifestlerini sed ile render edip apply eder.

5. Monitoring ve Performans

5.1 Prometheus Metrikleri

Metrik	Tur	Aciklama
http_requests_total	Counter	Endpoint/method/status bazli sayim
http_request_duration_seconds	Histogram	p50/p95/p99 latency

5.2 Grafana Dashboard (3 panel)

- Request Rate: `rate(http_requests_total[1m])` by endpoint
- Error Rate: 4xx/5xx oranlari, status koduna gore
- Latency p95/p99: `histogram_quantile` ile hesaplanir

5.3 k6 Sonuclari

Test	VU	Sure	p(95)	Hata	Sonuc
Smoke	3	15s	~5 ms	0%	PASS
Load	10->20	60s	~285 ms	0%	PASS

Yorum: p(95)=285 ms hedef olan 500 ms in cok altinda. En yavas endpoint ler /register ve /login — bcrypt password hashing guvenlik gereksinimi, optimize edilmez. Detayli rapor: perf/report.md.

6. Sonuc ve Ogrendiklerim

Olcut	Deger
Toplam Test	34 (8 unit + 18 integration + 3 TC + 5 E2E)
Test Coverage	%94
p(95) Latency	~285 ms (k6 load)
Docker Image	Multi-stage backend + NGINX alpine frontend
K8s Manifest	ConfigMap + Postgres + Backend + Frontend (4 dosya)
Grafana Panel	3 panel (rate, errors, latency)
CI Job	4 (lint, test, build, deploy-smoke)

6.1 Zorluklar

- **Frontend ayrimi:** Ilk versiyonda Jinja2 ile FastAPI icine gomuluydu — bu projede NGINX uzerinden static serve edilen tamamen ayri bir servis olarak yeniden tasarlandi. Bu, mikroservis mimarilerinin temel ayrimini ogretti.
- **Testcontainers Windows encoding:** psycopg2, Windows Turkce locale ile uyumlu calismadi. Cozum: pytestmark ile platforma ozel skip, Linux/CI da otomatik calisir.
- **K8s image render:** Manifestlerde imaj etiketi GitHub SHA degisken — sed ile placeholder degistirme calisti.

6.2 Ileride

- Helm chart paketleme (bonus +5)
- OpenTelemetry tracing (bonus +5)

- Connection pooling (PgBouncer) — yuksek yukte daha az overhead

7. Kaynaklar

Kaynak	URL
FastAPI	fastapi.tiangolo.com
Testcontainers Python	testcontainers-python.readthedocs.io
k6	k6.io/docs
Playwright Python	playwright.dev/python
Kubernetes	kubernetes.io/docs
Prometheus Python Client	github.com/prometheus/client_python